

第 6 次作业

算法设计与分析

姓名	倪伟丰	学号	2024110089
班级	24 大数据 2 班	教师	韩恺
提交日期	2026 年 4 月 18 日		

1 Problem 1

Suppose $T(n) \leq T(\lfloor \alpha n \rfloor) + T(\lfloor \beta n \rfloor) + n$ and $T(1) = 0$. Prove the following statements:

1.1

$T(n) = O(n)$, when $\alpha + \beta < 1$;

解答:

要证明 $\alpha + \beta < 1$ 时, $T(n) = O(n)$, 我们可以使用归纳假设的方法来分析。

我们这里证明一个比 $T(n) = O(n)$ 更强的结论:

$\exists c > 0$, 使得对于所有 $n \geq 1$, 都有 $T(n) \leq cn$ 。

基础情况: 当 $n = 1$ 时, $T(1) = 0 \leq c \cdot 1$, 基础情况成立。

归纳假设: 假设对于所有 $m < n$, 都有 $T(m) \leq cm$ 。

归纳步骤: 根据递归关系, 我们有:

$$\begin{aligned}T(n) &\leq T(\lfloor \alpha n \rfloor) + T(\lfloor \beta n \rfloor) + n \\&\leq c\lfloor \alpha n \rfloor + c\lfloor \beta n \rfloor + n \\&\leq c(\alpha n + \beta n) + n \\&= c(\alpha + \beta)n + n \\&= (c(\alpha + \beta) + 1)n\end{aligned}$$

因此, 只要:

$$(c(\alpha + \beta) + 1)n \leq cn$$

即:

$$c \geq \frac{1}{1 - (\alpha + \beta)}$$

所以, 我们可以选择 c 满足上述不等式, 从而保证 $T(n) \leq cn$ 对所有 $n \geq 1$ 成立。

因此, 当 $\alpha + \beta < 1$ 时, $T(n) = O(n)$ 。

1.2

$T(n) = O(n \log n)$, when $\alpha + \beta = 1$.

解答：

要证明 $\alpha + \beta < 1$ 时, $T(n) = O(n)$, 我们可以使用归纳假设的方法来分析。

我们这里证明一个比 $T(n) = O(n)$ 更强的结论：

$\exists c > 0$, 使得对于所有 $n \geq 1$, 都有 $T(n) \leq cn \log n$ 。

基础情况：当 $n = 1$ 时, $T(1) = 0 \leq c \cdot 1 \cdot \log 1 = 0$, 基础情况成立。

归纳假设：假设对于所有 $m < n$, 都有 $T(m) \leq cm \log m$ 。

归纳步骤：根据递归关系, 我们有：

$$\begin{aligned} T(n) &\leq T(\lfloor \alpha n \rfloor) + T(\lfloor \beta n \rfloor) + n \\ &\leq c \lfloor \alpha n \rfloor \log \lfloor \alpha n \rfloor + c \lfloor \beta n \rfloor \log \lfloor \beta n \rfloor + n \\ &\leq c(\alpha n) \log(\alpha n) + c(\beta n) \log(\beta n) + n \\ &= c(\alpha n)(\log n + \log \alpha) + c(\beta n)(\log n + \log \beta) + n \\ &= c(\alpha + \beta)n \log n + c(\alpha \log \alpha + \beta \log \beta)n + n \\ &= cn \log n + c(\alpha \log \alpha + \beta \log \beta)n + n \end{aligned}$$

因此, 只要：

$$cn \log n + c(\alpha \log \alpha + \beta \log \beta)n + n \leq cn$$

即：

$$c \geq \frac{1}{\alpha \log \alpha + \beta \log \beta + 1}$$

所以, 我们可以选择 c 满足上述不等式, 从而保证 $T(n) \leq cn \log n$ 对所有 $n \geq 1$ 成立。

因此, 当 $\alpha + \beta = 1$ 时, $T(n) = O(n \log n)$ 。

2 Problem 2

Suppose that we divide n elements into $\lfloor n/r \rfloor$ groups of r elements each, and use the median-of-medians of these $\lfloor n/r \rfloor$ groups as the pivot.

- (a) Show that the worst-case running time is $O(n)$, when $r = 7$;
- (b) Explain why the worst-case running time is not $O(n)$, when $r = 3$.

2.1

解答：

题干中的算法是把 n 个元素分成 $\lfloor n/r \rfloor$ 组，每组 r 个，求每组的中位数；再递归地求这些组中位数的中位数，作为 pivot；最后，用 pivot 做一次 partition，再递归到目标一侧。

因为 r 是常数，所以，求每组的中位数的时间是 $n \cdot T(r) = O(n)$ ，partition 成本也是 $O(n)$ 。

那么关键就在于 partition 之后子问题的规模。

我们设算法的总时间复杂度为 $T(n)$ ：

分成 $\lfloor n/r \rfloor$ 组，每组 r 个元素，求每组的中位数的时间是 $O(n)$ ，求这些中位数的中位数的时间是 $T(\lfloor n/r \rfloor)$ ，partition 的时间是 $O(n)$ 。

在 partition 之后，我们需要递归到被 pivot 划分后的两个部分中的一个部分，接下来我们要求到任意一侧的子问题规模的上界：

pivot 是 $\lfloor n/r \rfloor$ 个中位数的中位数，所以，我们至少能找到 $\lfloor n/r \rfloor / 2$ 个组，其中它们的中位数不小于 pivot

对于其中的每一个组，我们都能找到 $\lceil r/2 \rceil$ 个元素不小于 pivot，由于问题里的 r 是奇数，我们都能至少找到 $\frac{r+1}{2}$ 个元素不小于 pivot

去掉 pivot 本身，我们就能找到 $\frac{r+1}{2} \cdot (\lfloor \frac{n}{r} \rfloor - 1)$ 个元素不小于 pivot

那么，partition 之后被 pivot 划分后的两个部分中较大的一个部分的规模至多是：

$$n - \frac{r+1}{2} \cdot \left(\frac{\lfloor n/r \rfloor}{2} - 1 \right) \leq n - \frac{r+1}{4r}n + O(r) = \left(1 - \frac{r+1}{4r} \right)n + O(r)$$

所以，我们可以写出 $T(n)$ 的递归式：

$$T(n) \leq T(\lfloor \frac{n}{r} \rfloor) + T\left(\left(1 - \frac{r+1}{4r}\right)n\right) + O(n)$$

当 $r = 7$ 时：

递推式为：

$$T(n) \leq T(\lfloor \frac{n}{7} \rfloor) + T\left(\frac{5}{7}n\right) + O(n)$$

由作业的第一个问题的结论可知， $T(n) = O(n)$ 。

所以，当 $r = 7$ 时，算法的最坏情况时间复杂度是 $O(n)$ 。

当 $r = 3$ 时：递推式为：

$$T(n) \leq T(\lfloor \frac{n}{3} \rfloor) + T\left(\frac{2n}{3}\right) + O(n)$$

由作业的第一个问题的结论可知， $T(n) = O(n \log n)$ 。

所以，当 $r = 3$ 时，算法的最坏情况时间复杂度不是 $O(n)$ 。

3 Problem 3

You are interested in analyzing some hard-to-obtain data from two separate databases. Each database contains n numerical values — so there are $2n$ values total — and you may assume that no two values are the same. You'd like to determine the median of this set of $2n$ values, which we will define here to be the n^{th} smallest value.

However, the only way you can access these values is through queries to the databases. In a single query, you can specify a value k to one of the two databases, and the chosen database will return the k^{th} smallest value that it contains. Since queries are expensive, you would like to compute the median using as few queries as possible.

Give an algorithm that finds the median value using at most $O(\log n)$ queries.

解答：

设两个数据库中的元素按从小到大分别为

$$A_1 < A_2 < \cdots < A_n, \quad B_1 < B_2 < \cdots < B_n.$$

其中，查询数据库 1 的第 k 小元素即得到 A_k ，查询数据库 2 的第 k 小元素即得到 B_k 。

由于总共有 $2n$ 个互不相同的元素，而偶数个元素的中位数定义为中间两个元素的平均值，因此本题要求的是这 $2n$ 个元素中第 n 小元素与第 $n+1$ 小元素的平均值。

设合并后最小的前 n 个元素中，有 i 个来自数据库 1，则必有 $n-i$ 个来自数据库 2。若这个划分正确，则必须满足

$$A_i < B_{n-i+1}, \quad B_{n-i} < A_{i+1}.$$

这是因为：

- $A_1, \dots, A_i$ 和 $B_1, \dots, B_{n-i}$ 一共恰好有 n 个元素；
- 条件 $A_i < B_{n-i+1}$ 保证数据库 2 中剩余元素都比 A_i 大；
- 条件 $B_{n-i} < A_{i+1}$ 保证数据库 1 中剩余元素都比 B_{n-i} 大。

因此，合并后最小的前 n 个元素恰好是

$$A_1, \dots, A_i \quad \text{和} \quad B_1, \dots, B_{n-i}.$$

所以：

- 第 n 小元素为

$$\max(A_i, B_{n-i});$$

- 第 $n+1$ 小元素为

$$\min(A_{i+1}, B_{n-i+1}).$$

故中位数为

$$\frac{\max(A_i, B_{n-i}) + \min(A_{i+1}, B_{n-i+1})}{2}.$$

为了统一边界情况，定义

$$A_0 = B_0 = -\infty, \quad A_{n+1} = B_{n+1} = +\infty.$$

接下来对 $i \in [0, n]$ 做二分查找。

Algorithm 1 利用二分查找求两个数据库合并后的中位数

```

1:  $low \leftarrow 0, high \leftarrow n$ 
2: while  $low \leq high$  do
3:    $i \leftarrow \lfloor (low + high)/2 \rfloor$ 
4:    $j \leftarrow n - i$ 
5:   if  $i = 0$  then
6:      $a_L \leftarrow -\infty$ 
7:   else
8:     在数据库 1 中查询第  $i$  小的元素, 记为  $a_L$ 
9:   end if
10:  if  $i = n$  then
11:     $a_R \leftarrow +\infty$ 
12:  else
13:    在数据库 1 中查询第  $i + 1$  小的元素, 记为  $a_R$ 
14:  end if
15:  if  $j = 0$  then
16:     $b_L \leftarrow -\infty$ 
17:  else
18:    在数据库 2 中查询第  $j$  小的元素, 记为  $b_L$ 
19:  end if
20:  if  $j = n$  then
21:     $b_R \leftarrow +\infty$ 
22:  else
23:    在数据库 2 中查询第  $j + 1$  小的元素, 记为  $b_R$ 
24:  end if
25:  if  $a_L < b_R$  then
26:    if  $b_L < a_R$  then
27:       $x \leftarrow \max(a_L, b_L)$ 
28:       $y \leftarrow \min(a_R, b_R)$ 
29:      return  $(x + y)/2$ 
30:    else
31:       $low \leftarrow i + 1$ 
32:    end if
33:  else
34:     $high \leftarrow i - 1$ 
35:  end if
36: end while

```

正确性说明:

设当前二分枚举到 i , 并令 $j = n - i$ 。

- 若

$$A_i < B_{j+1}, \quad B_j < A_{i+1},$$

则 $A_1, \dots, A_i$ 与 $B_1, \dots, B_j$ 恰好构成合并后最小的 n 个元素，因此第 n 小元素就是

$$\max(A_i, B_j).$$

同时，剩余元素中最小的那个就是第 $n+1$ 小元素，因此

$$\text{第 } n+1 \text{ 小元素} = \min(A_{i+1}, B_{j+1}).$$

所以中位数为

$$\frac{\max(A_i, B_j) + \min(A_{i+1}, B_{j+1})}{2}.$$

- 若

$$A_i > B_{j+1},$$

说明 A_i 太大，即从数据库 1 中取的元素过多，因此正确答案对应的 i 一定更小，应令

$$high \leftarrow i - 1.$$

- 若

$$B_j > A_{i+1},$$

说明从数据库 1 中取的元素过少，因此正确答案对应的 i 一定更大，应令

$$low \leftarrow i + 1.$$

因此算法始终在正确区间内进行二分，最终一定能找到正确划分并返回正确的中位数。

复杂度分析：

每轮二分只进行常数次查询，最多查询 4 个值；二分查找共进行 $O(\log n)$ 轮。因此总查询次数为

$$O(\log n).$$

故该算法满足题意。

4 Problem 4

You are given an $n \times n$ grid graph G . (An $n \times n$ grid graph is just the adjacency graph of an $n \times n$ chessboard. To be completely precise, it is a graph whose node set is the set of all ordered pairs of natural numbers (i, j) , where $1 \leq i \leq n$ and $1 \leq j \leq n$; the nodes (i, j) and (k, l) are joined by an edge if and only if $|i - k| + |j - l| = 1$.)

Each node v of G is labeled by a real number x_v ; you may assume that all these labels

are distinct. A node v of G is a local minimum if the label x_v is less than the label x_w for all nodes w that are joined to v by an edge. The labeling is only specified in the following implicit way: for each node v , you can determine the value x_v by probing the node v . Show how to find a local minimum of G using only $O(n)$ probes to the nodes of G . (Note that G has n^2 nodes.)

You must show why your procedure uses $O(n)$ and why it is correct, i.e., that it always finds a local minimum.

解答：

我们用分治法来做。核心想法是：

- 先探测当前子网格的“中间十字架”：中间一行和中间一列；
- 再结合一个已经知道比当前子网格边界都小的结点 p ；
- 在这些候选点里找最小者 v ；
- 如果 v 已经是局部极小值，则直接返回；
- 否则， v 一定有一个更小的相邻点 u ，并且 u 必然落在四个象限中的某一个里；
- 递归到包含 u 的那个象限中继续寻找。

这样每一层只需探测一个“十字架”，即 $O(m)$ 个点（其中 m 是当前子网格边长），而下一层规模至多减半，因此总探测次数满足

$$T(m) = T(\lceil m/2 \rceil) + O(m),$$

从而

$$T(n) = O(n).$$

递归子问题的约定：

对一个边长为 m 的方形子网格 R ，我们递归调用

$$\text{FINDLOCAL}(R, p),$$

其中 $p \in R$ 是一个已经探测过的结点，并满足：

除 p 自身外， R 的所有边界结点的标号都严格大于 x_p 。

初始时，我们先探测整个 $n \times n$ 网格的外边界上所有结点（共 $4n - 4 = O(n)$ 个），取其中最小者作为 p 。显然此时上述条件成立。

算法伪代码如下：

Algorithm 2 在 $n \times n$ 网格图中寻找局部极小值

```

1: 过程 MAIN
2: 探测整个网格外边界上的所有结点
3: 令  $p$  为外边界上标号最小的结点
4: return FINDLOCAL( $1, n, 1, n, p$ )

5: 过程 FINDLOCAL( $top, bottom, left, right, p$ )
6: 令  $m = bottom - top + 1$ 
7: if  $m = 1$  then
8:   return  $p$ 
9: end if
10:  $r \leftarrow \lfloor (top + bottom)/2 \rfloor$ 
11:  $c \leftarrow \lfloor (left + right)/2 \rfloor$ 
12: 探测当前子网格中第  $r$  行的所有结点  $(r, j)$ , 其中  $left \leq j \leq right$ 
13: 探测当前子网格中第  $c$  列的所有结点  $(i, c)$ , 其中  $top \leq i \leq bottom$ 
14: 在结点集合

```

$$\{p\} \cup \{(r, j) : left \leq j \leq right\} \cup \{(i, c) : top \leq i \leq bottom\}$$

中取标号最小的结点, 记为 v

```

15: 探测  $v$  的所有相邻结点 (至多 4 个)
16: if  $x_v$  小于  $v$  的所有相邻结点的标号 then
17:   return  $v$ 
18: else
19:   令  $u$  为  $v$  的一个相邻结点, 且  $x_u < x_v$ 
20:   if  $u$  位于左上象限 then
21:     return FINDLOCAL( $top, r - 1, left, c - 1, u$ )
22:   else if  $u$  位于右上象限 then
23:     return FINDLOCAL( $top, r - 1, c + 1, right, u$ )
24:   else if  $u$  位于左下象限 then
25:     return FINDLOCAL( $r + 1, bottom, left, c - 1, u$ )
26:   else
27:     return FINDLOCAL( $r + 1, bottom, c + 1, right, u$ )
28:   end if
29: end if

```

正确性证明:

我们证明递归过程中始终保持如下不变式:

在调用 FINDLOCAL(R, p) 时, 除 p 自身外, 子网格 R 的所有边界结点的标号都严格大于 x_p 。

初始情形：在最开始的整个网格中，我们取外边界最小结点作为 p ，因此不变式显然成立。

递归步骤：考虑某次调用 $\text{FINDLOCAL}(R, p)$ 。设当前子网格的中间十字架（中间行与中间列）上的最小结点与 p 一起比较后，得到最小者为 v 。

- 若 v 比它所有相邻结点都小，则 v 就是局部极小值，算法正确返回。
- 否则，存在某个相邻结点 u 满足 $x_u < x_v$ 。由于 v 是“十字架 $\cup \{p\}$ ”中的最小者，所以：

$$x_u < x_v \leq x_p,$$

并且 u 不可能落在中间行或中间列上，因此 u 必然位于四个象限中的某一个。

设 Q 为包含 u 的那个象限。我们说明递归到 Q 是安全的。

象限 Q 的边界结点只可能来自两部分：

1. 当前子网格 R 的边界；
2. 当前的中间行或中间列。

对于第 1 类，由递归不变式知，除 p 外它们都大于 x_p ；而又有 $x_u < x_v \leq x_p$ ，故这些边界结点都大于 x_u 。

对于第 2 类，因为 v 是十字架上的最小结点，所以十字架上所有结点的标号都不小于 x_v ，于是也都严格大于 x_u 。

因此， Q 的所有边界结点（除可能的 u 自身外）都严格大于 x_u ，递归不变式在下一层仍然成立。

接着说明： Q 中一定存在一个局部极小值。

从 u 出发，若 u 不是局部极小值，就走向一个更小的相邻结点；如此反复。由于标号互不相同，且每一步都严格下降，这个过程不可能无限进行，必然终止于某个结点 z 。终止时， z 没有更小的相邻结点，因此 z 是局部极小值。

另一方面，这个下降过程不可能走出象限 Q ：因为一旦想走出 Q ，就必须经过 Q 的边界，而我们已经证明了这些边界结点（除可能的起点 u 外）全都比 u 大，更不可能比后续路径上的结点更小。所以整个下降路径始终留在 Q 内。

故 Q 中一定包含一个局部极小值，递归到 Q 不会丢失答案。

基例：当子网格边长为 1 时，唯一的结点就是 p 。根据递归不变式，它的所有可能“越界相邻位置”都不可能对应更小值，因此该点必为局部极小值，直接返回正确。

综上，算法总能返回一个局部极小值。

复杂度分析：

设当前子网格边长为 m 。在一次递归调用中：

- 探测中间一行需要 m 次；
- 探测中间一列需要 m 次；

- 探测最小候选点 v 的相邻结点只需 $O(1)$ 次。

因此每层新增探测次数为 $O(m)$ 。而递归只进入一个象限，所以子问题边长至多为 $\lceil m/2 \rceil$ 。于是

$$T(m) = T(\lceil m/2 \rceil) + O(m).$$

展开可得

$$T(n) = O(n + n/2 + n/4 + \cdots) = O(n).$$

初始时探测整个外边界还需 $O(n)$ 次，所以总探测次数仍为

$$O(n).$$

因此，该算法用 $O(n)$ 次探测就能找到一个局部极小值。